This article was downloaded by: [Nanyang Technological University]
On: 28 April 2015, At: 16:57
Publisher: Taylor & Francis
Informa Ltd Registered in England and Wales Registered Number: 1072954 Registered office: Mortimer House, 37-41 Mortimer Street, London W1T 3JH, UK



ISSN: 1246-0125
Journal of Decision Systems
Volume 22 Number 2 April 2013
Editors-in-Chief
Frédéric Adam
Frada Burstein

Taylor & Francis
Taylor & Francis Group

Journal of Decision Systems
Publication details, including instructions for authors and subscription information:
http://www.tandfonline.com/loi/tjds20

Open problems, formal problems

Gert Jan Hofstede [a]
[a] Wageningen University, Dept. of Informatics , Dreijenplein 2, NL 6703 , Wageningen , HB , the Netherlands
Published online: 31 May 2012.

To cite this article: Gert Jan Hofstede (1995) Open problems, formal problems, Journal of Decision Systems, 4:2, 155-165, DOI: 10.1080/12460125.1995.10511645

To link to this article: http://dx.doi.org/10.1080/12460125.1995.10511645

PLEASE SCROLL DOWN FOR ARTICLE

Taylor & Francis makes every effort to ensure the accuracy of all the information (the "Content") contained in the publications on our platform. However, Taylor & Francis, our agents, and our licensors make no representations or warranties whatsoever as to the accuracy, completeness, or suitability for any purpose of the Content. Any opinions and views expressed in this publication are the opinions and views of the authors, and are not the views of or endorsed by Taylor & Francis. The accuracy of the Content should not be relied upon and should be independently verified with primary sources of information. Taylor and Francis shall not be liable for any losses, actions, claims, proceedings, demands, costs, expenses, damages, and other liabilities whatsoever or howsoever caused arising directly or indirectly in connection with, in relation to or arising out of the use of the Content.

This article may be used for research, teaching, and private study purposes. Any substantial or systematic reproduction, redistribution, reselling, loan, sub-licensing, systematic supply, or distribution in any form to anyone is expressly forbidden. Terms & Conditions of access and use can be found at http://www.tandfonline.com/page/terms-and-conditions

Downloaded by [Nanyang Technological University] at 16:57 28 April 2015

Open problems, formal problems

Gert Jan Hofstede

Wageningen University
Dept. of Informatics
Dreijenplein 2
NL 6703 HB Wageningen, the Netherlands

Downloaded by [Nanyang Technological University] at 16:57 28 April 2015

ABSTRACT. *This article's purpose is to introduce the notion of an open problem as opposed to that of a formal problem, and to indicate the importance of these notions for those who develop decision support systems. These two notions emerged out of a study on the feasibility of interactive planning systems. Typically, developers of automated information systems think in terms of formal problems while planners deal with their planning tasks as open problems. This leads to problems of communication. Adopting a participative development method in itself is not enough to overcome these problems. Explicit search for open aspects of the planners' problem, and explicit concern for them during DSS design, are also needed. Open aspects should be searched for and agreed upon. An important guideline for the design of a DSS is that if open aspects are found, it may be wise not to hard-wire them into a model, because that might rob the planners of the flexibility they need.*

RÉSUMÉ. *Cet article a pour but d'introduire le concept du problème ouvert et de l'opposer à celui du problème formel. Ensuite, l'importance de cette distinction pour le développement des SIAD est indiquée. La distinction permet de remédier aux problèmes de communication entre développeurs (qui aiment concevoir la planification en termes de problèmes formels) et planificateurs (qui traitent le planning comme un problème ouvert). L'article indique comment tenir compte des aspects ouverts durant les projets de développement des SIAD.*

KEY WORDS : *open problem, formal problem, user-developer communication, modelling.*
MOTS-CLÉS : *problème ouvert, problème formel, communication utilisateurs-développeurs, modelage.*

1. Introduction

When a decision support system, or indeed any automated information system, is developed in an organization, cooperation between computer specialists and people

from the user organization is necessary. This article is about the problems that may occur when these two types of people communicate. It is no secret that in the past, huge amounts of creativity, time and money were spent on information systems that were never used. Could this have to do with differences in perception? Could it be that computer specialists and users are not talking about the same thing when they refer to a planning problem?

The article is based on the author's doctoral research (Hofstede 1992). That research project included building a system to assist small enterprise managers in production planning. Failure to get the first system version adopted by the intended users induced the author to reconsider the role of formal models in an information system.

The present article is by no means the first one in which problems of communication between developers and users of DSS are addressed. Within the OR community, presumed excessive reliance on formal models has been widely criticized since Ackoff (1979). More recently, in the DSS community, many authors have pleaded for participative, evolutionary development: Courbon and Bourgeois (1980), Keen (1986), Alavi and Napier (1989), Angehrn and Lüthi (1990), Gould et al. (1991), to name but a few.

The article is merely a statement about a theoretical position. As such it treads in the footsteps of an excellent article by Landry et al. (1985) who propose to change the concept of 'a problem' as it was then, and still is, used in the DSS world from an objective reality into a subjective representation, 'someone's problem'.

Much of the justification of what is put forward here, both based on extensive literature studies and on the author's experience as a DSS developer, can be found in Hofstede (1992). That book also discusses practical implications for DSS development. A book fully aimed at practitioners (Hofstede et al. 1995) is in print at the time of writing of this article.

2. A planner's problem is an open problem

What is an *open problem*? Let us take the example of an ubiquitous type of problem that would at first sight appear to be clear-cut and for which sufficient empirical data are available. Production planning at a factory in which orders for batches of different products have to be allotted to various production lines is such a type of problem. Typically, a plan is made daily by an experienced planner, possibly a team of planners. Data required are on the one hand the order list, on the other the availability of raw materials, machines, and personnel.

An open problem, then, is an instance of a planner's planning problem which does not present itself to the planner as a clear-cut puzzle. Rather, part of the planner's work is to decide what the problem is. Open aspects of the problem are all those aspects about which the planner has to decide at planning time. Speaking with Landry et al. (1985), openness is a property of a planner's problem, not of a planning problem.

Downloaded by [Nanyang Technological University] at 16:57 28 April 2015

There are four reasons why a planner would handle his planning task as an open problem. The first is that the problem can be *volatile*. If it is, there is no single moment in time when the planner can say by any objective standard "Now I have all the data and I can start solving the planning problem for tomorrow." Empirical evidence that this claim is true was given by McKay et al. (1989) who investigated over 300 production planners and found, for instance, that in none of the planning situations the quality, quantity, and arrival times of the materials were known in advance. In practice this means that the longer a planner postpones his planning activities, the more data will be at his disposal. It is essentially his own decision when to decide not to wait or ask for new data any longer. Even then, new data may come in while he is planning and overrule the initial data.

The second open aspect of a production planner's problem is that it may contain unpredictable uncertainty. McKay et al. in their survey found that not one of their 300 cases possessed predictable and reliable set-up and processing times, nor reliable, stable and predictable forecasts of product demand. In other words: many of the planner's data are at best educated guesses. Incidentally, uncertainty as such does not constitute openness. It can be modelled into a DSS if its nature can be predicted. Unpredictability, however, forces the planner to take ad hoc decisions about the problem at the time of planning.

In the third place, a planner's problem may be open when people are affected by the plans. In most organizations, people cannot be treated like any other means of production. Their interests carry special weight which a planner had better take into account. A relation of give and take usually exists between the planner and the planned personnel.

Fourthly, the production planning problem is not the same each time it occurs. It is *unstable*. From day to day, changes take place in the organization that have an impact upon the rules the planner must apply. McKay et al. remark that none of their cases had a well-known and stable manufacturing process. One can also think of new employees, new machines, new products, new working hours, or of occasional interruptions for a variety of reasons.

So, even a comparatively well-defined type of planning problem turns out to be an open problem to the planner in up to four ways. Volatility, unpredictable uncertainty, people's interests and lack of stability call for ad hoc decisions by the planner that constitute part of his planning activities.

Incidentally, the reader will appreciate the difference between the notion of an *open* problem and that of an *unstructured* problem (see e.g. Mintzberg et al., 1976). 'Unstructured' refers to the internal clarity of the situation at hand, whereas 'open' refers to its delineation. By careful study, e.g. during a DSS development problem, it is possible to structure a previously unstructured problem. The same careful study may also show that there are open aspects to the planning situation, but it cannot take

Downloaded by [Nanyang Technological University] at 16:57 28 April 2015

these away. If, for instance, an experienced production planner applies a fixed routine for tackling the daily planning task, this implies he has structured the problem. It may, however, still be very open and require ad hoc estimates about orders, machines, materials, or workers.

Summarizing, we see that the concept of an open problem has meaning in the context of actual problem solving, in our case by planners. A planner deals with a planning problem as an open problem if his planning activities include *deciding what the problem is*. In other words, an open problem is one the solver deals with not as a puzzle, but as a design problem.

Downloaded by [Nanyang Technological University] at 16:57 28 April 2015

3. A developer's planning problem is a formal problem

Let us assume that a DSS project is started in an organization, with the aim to build a system to help the production planners. Figure 1. shows which roles have to be played during the project. Its purpose is to visualize the many actors, with as many different perspectives, that may exist during the project's life. The roles of client and judge are crucial, because whoever plays them can stop the project. The users, in our case the production planners, may or may not also play other roles, e.g. be part of the project team or act as domain experts. The figure shows roles, not people, and people can play multiple roles.

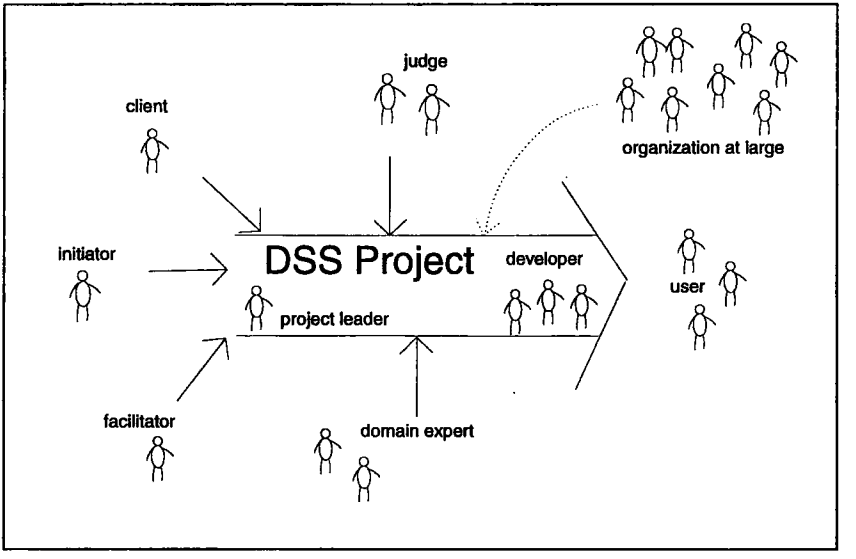


Figure 1. *Roles in a project (inspired by Checkland and Scholes 1990, figure 1.2)*

We shall now concentrate on the developers' role. Most likely, the developers will build models of the planning situation. To do this, they will try and objectively identify what they perceive to be the object of their project, i.e. the object system. Depending on the project approach, the planners may be more or less intensively involved in the modelling activities.

Singling out a decision situation is a powerful tool for analysis. Once a line is drawn around an object system, ambiguity is eliminated. By virtue of this, various types of models of the object system can be drawn. Most of these models are formal in the sense that *their syntax is unambiguous*. Specification of models in a formal language in turn opens the door to the application of a wealth of tools from information technology. Present-day organizations would collapse without information technology and the concepts of system and model. To be more precise, information systems are widely built on the basis of formal specifications in the form of various models. Communication between automated systems is achieved through formal standards. Simulation modelling is an enormously powerful tool for non-destructive experimentation in a wide variety of disciplines.

According to Hofstede (1992, p. 43) most research schools that build models of planning situations employ similar model components. These are:

1. A 'world model' or a description of relevant dimensions, be they variables (including time) or objects.
2. A description of the limitations or constraints on the values of the dimensions, on actions or on combinations of values.
3. A description of actions (these are often called 'assignments' in the sense in which this word is used in programming languages: allotting a value to a variable). These actions change an unplanned world of variables or objects into a plan.
4. A description of a strategy for using the actions to solve the problem, that is, attain the goals.
5. A description of goals, or quality criteria, in terms of the dimensions.

DSS developers tend to feel strongly about the necessity of employing these model components in a formal manner, as the following anecdote will illustrate. While presenting a DSS at a conference, the author was once asked "Do you have a goal function?" His reply to the negative met with the rejoinder "Then you do not have a problem". To which he replied that the planners for whom the DSS was being made certainly did have a planning problem, goal function or no goal function!

This anecdote brings to light the price that must be paid for the use of formal concepts in DSS design. This price is precisely the strength of the approach: it is the line drawn around the object of study, singling it out from the rest of the world. As soon as this line is drawn the object system begins a life of its own in the most literal sense of the word. With Landry et al. we should speak not of 'the' object system, but of the DSS developers' object system. It becomes a *formal problem*, an entity

Downloaded by [Nanyang Technological University] at 16:57 28 April 2015

known to those who drew the system boundaries. It will only be meaningful to the planners in the organization it is ultimately intended to serve if these people draw the same line, or if they can at least perceive it. And even they do, it may prevent them from dealing with open aspects of their planning work in a way they find satisfactory.

The danger of isolating a decision problem from its context may be illustrated by taking the Garbage Can model of decision making in organizations (March and Olsen, 1976) as a reference. In the Garbage Can view, decisions result from the coincidental meeting of problems, solutions, participants and choice opportunities. On such an assumption, an analysis of one isolated problem can clearly yield no more than very partial information on the Garbage Can's behaviour. Another danger is that lack of stability is not taken into account: processes that occur in the organization over time and that affect the planning situation, such as learning and change, are not expressed in the formal problem formulation. This is e.g. eloquently pointed out by Checkland and Casar (1986).

In conclusion, we see that a formal problem is the interpretation of (in our case) a planning situation by a DSS developer or project team in terms of unambiguous models. It may be highly parameterizable, but it has no loose ends.

4. What can go wrong in a DSS project?

Can formal models be used in planning system development for situations which planners deal with as open problems? In other words, how suitable are formal problems for modelling open problems? Is a formal problem drawn up by a DSS specialist still the same problem as the one the planners are faced with?

A literature study of numerous cases of planning system development (Hofstede 1992, chapter 2) has convinced the author that, with exceptions, developers of DSS systematically neglect open problem aspects. This results in the building of large numbers of sophisticated planning systems that can adequately solve the formal problems they were based on. However, a vast majority of these systems are apparently not very helpful in solving the planners' problem they were intended for, for they are not adopted by the intended users.

What happens? Let us draw a black and white picture for the sake of clarity. The developers (including both model builders and software builders) have not solved the planning problem they were hired to solve, but created their own formal problem and a DSS to solve that problem. For lack of a common understanding, neither developers nor users can prevent *nor even notice* this. The users cannot express what it is that makes the system unuseable, and they blame the developers. The developers do not perceive what is wrong with their method, and blame the users.

It is worthwhile to stress that this misunderstanding need not at all be caused by a lack of commitment, or willingness to cooperate, on either side. It is a problem of perception. This has an important implication. Even the adoption of a highly

Downloaded by [Nanyang Technological University] at 16:57 28 April 2015

participative development method may not be adequate in overcoming the misunderstanding, as long as the openness of parts of the planners' problem is not explicitly recognized and dealt with as a design issue.

To sum up, much energy has been lost in DSS projects because of differences in perception between developers and users, resulting in the building of systems that were not adopted. A DSS development team could gain both time and system quality by explicitly taking into account open aspects of the planners' problem.

Downloaded by [Nanyang Technological University] at 16:57 28 April 2015

5. Early focus on openness

The clue to improvement in the success rate of DSS projects lies in improving communication at an early stage in the project. Before they start to build formal models, developers and users have to establish meaningful communication. This recommendation in itself is by no means a novelty. As mentioned previously, an extensive literature exists about participative design for DSS projects. What is proposed here is that developers and users together pay *explicit attention to open aspects* of the planning situation at hand.

A first question is which method to adopt for the project. In this regard we note that a DSS development team which attempts to find 'the' objective problem is not likely to appreciate the difference between the planners' and the developers' problem. Therefore, we would advocate an approach that is both participative and allows for explicit recognition of different viewpoints. As Landry et al. (1985) point out, Checkland's Soft Systems Methodology (Checkland and Scholes, 1990) might well be used within the project. An information systems development method based on Checkland's ideas is Multiview (Avison and Wood-Harper, 1990).

Whichever method is chosen is bound to contain, early in the project, an activity known as 'problem formulation', 'analysis', 'problem finding', or by a similar name. It is during this activity that openness can be addressed. Developers and users together can establish a rough model of the planning problem, e.g. with help of the quintuple dimensions, constraints, actions, strategy and goals introduced previously. Then, and most importantly, they establish for each element of the model *who decides upon it at which moment in time.* Most likely, the dimensions (orders, machines, workers, materials, etc) are known at design time. But who decides about the constraints, and who are involved in setting the goals?

Two outcomes will be generated by this. The first is an insight into who owns parts of the planning problem. Most likely there are several owners: planners, personnel manager, production line managers, etc. Secondly, and most significantly for the argument of this article, an overview will result of decisions that are taken at the last minute by the planner. These will most likely be far more numerous and far-reaching than the developers would have expected them to be. They constitute the open aspects of the planner's problem.

With this result, a discussion can be started about both the desirability of the current state of affairs and the feasibility of making changes to it. Those aspects that remain open when the DSS is in operation can be explicitly taken into account in the design of the DSS.

6. Open problems: to model or not to model

Let us suppose that a number of open aspects were found during problem finding and that it was concluded they should remain so. For the DSS, this means that the planner must be able to take decisions about these aspects during planning and incorporate these decisions in the automated plans the DSS produces.

According to Hofstede et al. (1995; or see Verbeek, 1991) a DSS for planning may include up to six functions:

1 Manipulate data
2 Represent a plan
3 Manipulate a plan
4 Evaluate a plan
5 Generate a plan
6 Manage plans and scenarios

The first and sixth functions are administrative ones, necessary in all DSS. The remaining four deal with plans specifically.

How can open problem aspects be taken into account in the DSS, once they are recognized as such by the project team? If they are to remain open, the planners must be able to deal with them as such each time they are planning, and decisions cannot be hard-wired into the DSS at the time of its development.

Basically, there are two possibilities. The first option is to model the open aspects. They are then incorporated in the models for all the functions of the DSS. This is a great challenge for the developers, because the user must at all times both see what the model is doing and be able to overrule it. Hence, visual interactive modelling is needed (see e.g. Angehrn and Lüthi, 1990, for a motivating example). Many modelling techniques, however, preclude effective interaction, because they do not allow the user to meddle with the search space they are exploring. Allowing the user to alter parameters after a run of the model and then to run it again is usually not a solution. The user needs to be able to make fine-grained modifications in a partial plan. Finding modelling techniques that combine algorithmic power with user empowerment is still a research area in which much can be discovered.

The second option is to lower the level of ambition from 'modelling the planners' problem' to 'modelling *support for* the planners' problem'. What the planner needs to see is those data, or that partial model, that will allow him to make the best possible decisions about the open problem aspects. This means great emphasis will be put on adequate interfaces for representing plans and related data. Much of the

Downloaded by [Nanyang Technological University] at 16:57 28 April 2015

modelling for generating plans is transferred from the automated models to the planners' mental activities.

Although less ambitious from a developer's point of view, this second option places high demands on the quality of communication within the project team. Also, it does not come easy to most developers to identify a problem aspect and then leave it out of the formal models that he develops, and let the planner resolve it manually.

A brief example may serve to illustrate the importance of the choice to model or not to model open problem aspects. It is discussed at length in Hofstede (1992). The example is about a DSS for production planning at potted plant nurseries. The main problem is to utilize available space as fully as possible, which entails that plants have to be moved about the greenhouse repeatedly as they grow. Labour and market considerations also play a role. The initial aim of the project was to come up with a visual interactive interface based on a backtracking algorithm. The function 'generate a plan' of the resulting system could be used in three ways:

- 'Automat': use the backtracking algorithm to generate an entire plan. There were possibilities for interrupting the Automat to change parameters during a run.
- 'Step': use the algorithm, but proceed one production lot at a time. The planner could choose among various possibilities found by the algorithm, or overrule the system's proposals.
- 'Hand': do not use the algorithm. The planner could freely edit the plan.

At the end of the day, only the option 'Hand' was a success with the users. A major impediment to the success of 'Automat' and 'Step' was that these options did not allow the user to cheat on their formal models. For instance, users would typically produce plans with over 100% use of greenhouse space, or plans containing production lots with 'illegal' space requirement over time. The backtracking algorithm did not allow them to do such things, and was therefore rejected.

Returning to the six DSS functions, it can be concluded that the second and third are the most important ones for dealing with open problem aspects. The planner must be able to see a plan, partial or finished, and to manipulate it freely. The fourth and fifth function need not be performed by the DSS. Although they can add great power, there is a tradeoff when open problem aspects are at play. Model components needed to enable a DSS to evaluate, and especially to generate a plan, may get in the way of the manipulation function.

7. Conclusion

In this article the notion of an *open problem* is presented and contrasted with that of a *formal problem*. The context is that of production planning. An open problem is a planners' problem. It is the problem as it presents itself to the planners who have to resolve it as part of their work. Open aspects of their problem are those that the

Downloaded by [Nanyang Technological University] at 16:57 28 April 2015

planners decide about at the time of planning, not prior to it. This may be because of volatility (new data continually arrive), lack of predictability, interests of people, or lack of stability (the problem changes each time it occurs).

Formal problems are developers' problems. They are problems that are expresed in an unambiguous modelling language. The more open a problem is to a planner, the more problematic it becomes to represent it as a formal problem when building a DSS. In practice, precocious use of formal models in DSS projects can act as a barrier to usability of the resulting system.

It is argued that in the past, much effort in DSS projects has been ineffective due to unresolved differences in problem perception between developers and users. Although participative development improves cooperation and mutual understanding, it cannot take away these differences in perception unless the distiction between open and formal problems is brought to light. Regarding open aspects, planners should be left to 'draw the line', not models.

Recognizing open problems for what they are, and not equating them to formal problems, is not only relevant for the type of planning situations described in this article. On the contrary, there is ample evidence that any problem situation in organizational life is open to some extent and in at least some aspects. Therefore, awareness of the notion of an open problem will be beneficial to anyone studying organizational life, and particularly to DSS developers.

Acknowledgements

I wish to express my gratitude to the two anonymous reviewers whose comments caused me to reduce the article's length and (I hope) sharpen its focus.

Downloaded by [Nanyang Technological University] at 16:57 28 April 2015

8. References

Ackoff Russell L., 1979 : « The Future of Operations Research is Past », *Journal of the Operational Research Society* 30, pp. 93-104.

Alavi Maryam and Napier H. Albert, 1989 : « An experiment in applying the adaptive design approach to DSS development », in Sprague Ralph H. and Watson Hugh J. (eds), *Decision Support Systems: Putting theory into Practice*, 2nd edition, Prentice-Hall, London, pp. 82-92.

Angehrn Albert A. and Lüthi Hans-Jacob, 1990: « Intelligent Decision Support Systems: A Visual Interactive Aproach », *Interfaces* 20 no 6, pp. 17-28.

Avison D.E. and Wood-Harper A.T., 1990 : *Multiview, An Exploration in Information Systems Development*, Blackwell, Oxford.

Checkland Peter and Casar Alejandro, 1986 : « Vickers' concept of an appreciative system: a systemic account », *Journal of Applied Systems Analysis* 13, pp. 3-17.

Checkland Peter and Scholes Jim, 1990 : *Soft Systems Methodology in Action*, John Wiley, Chichester.

Courbon Jean-Claude and Bourgeois Marcel, 1980 : « The information system designer as a nurturing agent of a socio-technical process », in Lucas Henry C. Jr. et al. (eds), *The Information Systems Environment*, North-Holland, Amsterdam, pp. 139-148.

Gould John D., Boies Stephen J. and Lewis Clayton, 1991 : « Making usable, useful, productivity-enhancing computer applications », *Communications of the ACM* 34 no 1, pp. 74-85.

Hofstede Gert Jan, 1992 : *Modesty in Modelling: on the Applicability of Interactive Planning Systems*, Thesis Publishers, Amsterdam.

Hofstede G.J., Verbraeck A., Akkermans H.A., Bots P.W.G., Mietus D.M., Verbeek P.J. and De Jong R., 1995 : *De planner de baas*, Lansa Publishing, Rijswijk (In print. To be translated into English for 1996).

Keen Peter G.W., 1986 : « Decision Support Systems: The Next Decade », in McLean Ephraim R. and Sol Henk G. (eds), *Decision Support Systems: A Decade in Perspective*, North-Holland, Amsterdam, pp. 221-237.

Landry Maurice, Pascot Daniel and Briolat Dominique, 1985 : « Can DSS Evolve Without Changing Our View of the Concept of 'Problem'? », *Decision Support Systems* 1 pp. 25-36.

March James G. and Olsen Johan P., 1976 : *Ambiguity and Choice in Organizations*, Universitetsforlaget, Bergen.

McKay K., Buzacott J.A. and Safayeni F.R., 1989 : « The Scheduler's Knowledge of Uncertainty: the Missing Link », in *Knowledge Based Production Management Systems*. J. Browne (ed.), pp. 171-189, Elsevier, Amsterdam.

Mintzberg Henry, Raisinghani Duru and Théorêt André, 1976 : « The Structure of "Unstructured" Decision Processes », *Administrative Science Quarterly* 21, pp. 246-275.

Verbeek Peter J., 1991 : *Learning about Decision Support Systems: two cases on manpower planning in an airline*, Tinbergen Institute Research Series no. 7, Thesis Publishers, Amsterdam.

Downloaded by [Nanyang Technological University] at 16:57 28 April 2015